

Machine Learning Model Deployment Using FastAPI and Docker

Abstract

This project demonstrates how to deploy a trained machine learning model using FastAPI and Docker. The deployed API exposes inference endpoints that allow users to send input data and receive model predictions. Docker is used to package the application for portable deployment.

Introduction

Machine learning models are commonly developed in notebooks, but production systems require a reliable way to serve predictions. FastAPI provides a modern framework for creating REST APIs, while Docker ensures consistent execution across environments.

Objectives

1. Deploy a trained model using FastAPI.
2. Create prediction endpoints.
3. Containerize the application using Docker.
4. Demonstrate inference through API requests.

System Architecture

User → REST API (FastAPI) → Trained Model → Prediction Response.

The API receives JSON input, preprocesses the data, invokes the trained model, and returns predictions in JSON format.

Tools and Technologies

Python, FastAPI, Scikit-learn, Docker, Uvicorn, GitHub, REST APIs.

Project Structure

```
model-api/  
■■■ app.py  
■■■ model.pkl  
■■■ requirements.txt  
■■■ Dockerfile  
■■■ sample_request.json  
■■■ README.md  
■■■ screenshots/
```

API Design

Endpoint: POST /predict

Input: JSON features

Output: Prediction result and status message.

Sample Request

```
{ "feature1": 10, "feature2": 20, "feature3": 30 }
```

Sample Response

```
{ "prediction": 1, "status": "success" }
```

Docker Containerization

Docker packages the application, dependencies, and model into a portable image. This ensures the application behaves consistently across systems.

Commands

```
docker build -t ml-api .  
docker run -p 8000:8000 ml-api
```

Testing

```
curl -X POST http://localhost:8000/predict -H 'Content-Type: application/json' -d  
'{"feature1":10,"feature2":20,"feature3":30}'
```

Results

The deployed API successfully accepts requests and returns model predictions. Containerization simplifies deployment and distribution.

Advantages

- Easy deployment
- Scalable architecture
- Platform-independent execution
- Reusable API endpoints

Future Enhancements

- Deploy on cloud platforms
- Add authentication
- Implement monitoring and logging
- Use CI/CD pipelines

Conclusion

This project successfully demonstrates model deployment using FastAPI and Docker. The solution provides a production-ready workflow for serving machine learning predictions.